

Dokumentacja Ogólna

Projekt: Aplikacja webowa Taskly - Menedżer Zadań

Klasa: 3F

Skład zespołu i role:

- **Jakub Schmidt** - Scrum Master (Zarządzanie projektowe, organizacja sprintów, koordynacja prac)
- **Szymon Piotrkowski** - Analityk (Wymagania, makiety, przypadki użycia, User Stories)
- **Bartosz Słomski** - Programista (Implementacja w Vue.js, Local Storage)
- **Mateusz Witka** - Tester (Scenariusze testowe, weryfikacja wymagań, środowisko testowe)

1. Informacje ogólne i Cel aplikacji

Informacje ogólne

- **Nazwa aplikacji:** Taskly
- **Typ aplikacji:** Aplikacja webowa (menedżer zadań)
- **Technologie:** HTML, CSS (Tailwind CSS), JavaScript, Vue 3, GSAP, LocalStorage

Cel aplikacji

Projekt powstał z myślą o stworzeniu prostego narzędzia pomagającego w planowaniu i utrzymaniu porządku w codziennych obowiązkach. Aplikacja pozwala zarządzać zadaniami i filtrować je według kategorii i statusu. Taskly pozwala na tworzenie nowych wpisów, edycję, usuwanie oraz oznaczanie ich jako wykonane.

2. Wymagania użytkownika (User Stories)

Poniżej znajdują się user stories opracowane przez Szymona, które stanowiły podstawę do stworzenia logiki aplikacji i późniejszych testów.

US-01

Jako użytkownik chcę dodać nowe zadanie z tytułem i terminem, żeby wiedzieć co mam do zrobienia i kiedy.

Kryteria: Formularz ma pole na tytuł, kategorię i datę nie można dodać zadania bez tytułu, po dodaniu zadanie pojawia się na liście.

US-02

Jako użytkownik chcę oznaczyć zadanie jako ukończone, żeby wiedzieć co już zrobiłem.

Kryteria: Przy każdym zadaniu jest checkbox, po zaznaczeniu zadanie wygląda inaczej, można też je odznaczyć.

US-03

Jako użytkownik chcę edytować zadanie, które już dodałem, żeby poprawić błąd lub zmienić termin.

Kryteria: Jest przycisk edycji przy zadaniu; po kliknięciu edytuj wszystkie dane są możliwe do zmiany, po zapisaniu zadanie się aktualizuje.

US-04

Jako użytkownik chce usunąć zadanie z listy, żeby nie zaśmiecać listy starymi zadaniami.

Kryteria: Jest przycisk usuń przy zadaniu, po kliknięciu zadanie znika z listy.

US-05

Jako użytkownik chcę, żeby moje zadania były zapisane po zamknięciu przeglądarki, żeby nie tracić danych.

Kryteria: Dane są w localStorage po odświeżeniu strony zadania dalej są widoczne.

US-06

Jako użytkownik chcę filtrować listę zadań, żeby szybko znaleźć to, czego szukam.

Kryteria: Można filtrować po statusie (aktywne/ukończone) oraz po kategorii widać ile zadań jest aktualnie wyświetlanych.

3. Funkcjonalności i Opis interfejsu

Funkcjonalności systemu

- **Zarządzanie zadaniami:** Dodawanie nowych zadań, edytowanie istniejących, usuwanie wpisów oraz oznaczanie ich jako wykonane.
- **Kategorie:** Zadania można przypisać do kategorii "To-Do" lub "Codzienne"
- **Terminy:** Możliwość przypisania konkretnej daty oraz godziny wykonania.
- **Filtrowanie:** Lista zadań może być filtrowana według statusu (wszystkie, aktywne, ukończone) kategorii oraz terminu wykonania (zaległe, na dzisiaj, przyszłe).
- **Przechowywanie danych:** Automatyczny zapis w LocalStorage zapewnia zapisanie danych po odświeżeniu karty.

Opis interfejsu

- **Nagłówek:** Górna część strony zawierająca nazwę aplikacji oraz krótki opis przeznaczenia.
- **Panel tworzenia i edycji (Lewa strona):** Formularz pozwalający wpisać nazwę, wybrać kategorię, ustawić termin oraz zapisać zmiany lub anulować edycję. W przypadku braku tytułu pojawia się tu komunikat o błędzie.
- **Lista zadań (Prawa strona):** Wyświetla aktualne zadania. Każdy element ma checkbox statusu, nazwę, kategorię, termin oraz przyciski do edycji i usunięcia.

4. Architektura i Logika aplikacji

Plik HTML

Odpowiada za budowę struktury strony, rozmieszczenie formularzy, prezentację listy zadań oraz wyświetlanie filtrów. Wizualna część została wystylizowana przy użyciu narzędzia **Tailwind**, a dynamiczne wyświetlanie elementów i powiązanie danych zaimplementowano dyrektywami Vue (**v-model**, **v-if**, **v-for**).

Plik JavaScript (app.js)

Zawiera kompletną logikę programu. Odpowiada za obsługę formularza, filtrowanie, zapis/odczyt danych oraz wywoływanie animacji interfejsu.

Najważniejsze funkcje w kodzie:

- **loadTasks()** – Odczytuje zapisane dane tekstowe z LocalStorage i przekształca je na obiekt.
- **saveTasks()** – Aktualizuje stan bazy danych i zapisuje go w pamięci przeglądarki
- **submit()** – Obsługuje proces dodawania nowego zadania lub zapisywania zmian po edycji. Sprawdza też czy pole tytułu nie jest puste
- **toggleDone(id)** – Zmienia stan wykonania wybranego zadania.
- **deleteTask(id)** – Usuwa wskazany obiekt zadania z listy głównej
- **filteredTasks()** – Funkcja typu **computed**, która zwraca listę zadań spełniających aktualnie wybrane filtry

5. Wykorzystane technologie i Animacje

Tabela technologiczna

Technologia	Zastosowanie w projekcie
HTML	Budowa struktury strony i formularzy
Tailwind	Nowoczesne stylowanie i układ interfejsu
Vue 3	Dynamiczna logika aplikacji, reaktywność danych
GSAP	Płynne animacje elementów interfejsu
LocalStorage	Trwałe zapisywanie danych zadań w przeglądarce

Obsługa animacji

Animacje zaimplementowano za pomocą biblioteki GSAP oraz mechanizmu przejść wbudowanego w framework Vue. Podnoszą one wygodę użytkownika poprzez:

- Stopniowe i płynne pojawianie się elementów interfejsu po włączeniu aplikacji
- Animowane wchodzenie nowych zadań na listę oraz ich znikanie przy usuwaniu
- Efekt „shake” (potrząsanie) panelu formularza, jeśli użytkownik spróbuje wysłać puste pole
- Wizualne podświetlenie i wyróżnienie karty edycji, gdy modyfikujemy zadanie

6. Dokumentacja procesów testowych

Cel i zakres testów

Testy miały na celu sprawdzenie poprawności działania wszystkich opcji programu i upewnienie się, że aplikacja reaguje zgodnie z założeniami.

- **W zakresie testów znalazły się:** Działanie formularza, dodawanie, edycja, usuwanie zadań, checkbox statusu, filtry oraz poprawność działania LocalStorage.
- **Poza zakresem testów pozostawiono:** Wydajność przy tysiącach zadań, bezpieczeństwo przechowywanych danych, pełne dostosowanie mobilne oraz integrację z zewnętrznymi bazami danych (API).

Środowisko testowe

- **System operacyjny:** Windows 10
- **Przeglądarka:** Google Chrome
- **Narzędzia dodatkowe:** Vue DevTools, zakładka Application w oknie narzędzi deweloperskich przeglądarki.
- **Warunki wstępne:** Uruchomiona aplikacja, brak blokady zapisu LocalStorage.

Scenariusze i przypadki testowe

ID	Nazwa testu	Kroki wykonania	Dane wejściowe	Oczekiwany rezultat	Status
T1	Dodawanie zadania	1. Otwórz aplikację. 2. Wpisz nazwę. 3. Wybierz kategorię.	Tytuł: „Nauka” Kategoria: To-Do	Zadanie pojawia się na liście.	Zaliczony

		4. Zapisz.			
T2	Edycja zadania	1. Wybierz zadanie. 2. Kliknij edycję. 3. Zmień dane. 4. Zapisz.	Nowy tytuł: „Projekt”	Zadanie zostaje zaktualizowane.	Zaliczony
T3	Usuwanie zadania	1. Wybierz zadanie. 2. Kliknij przycisk usunięcia.	Istniejące zadanie	Zadanie znika z listy.	Zaliczony
T4	Oznaczanie jako wykonane	1. Zaznacz checkbox przy zadaniu.	Aktywne zadanie	Status zadania zmienia się na wykonany	Zaliczony

				(skreślenie tekstu).	
T5	Walidacja pustego formularza	1. Pozostaw pole tytułu puste. 2. Kliknij zapis.	Puste pole tekstowe	Wyświetlenie komunikatu błędu i animacja potrząsania.	Zaliczony

Wyniki testów i podsumowanie

- Liczba wykonanych testów: 5
- Zakończone sukcesem: 5
- Wykryte błędy: 0

Podczas testowania systemu nie wykryto żadnych krytycznych błędów funkcjonalnych, które wymagałyby natychmiastowej poprawy w kodzie.

7. Wnioski końcowe z realizacji projektu

Jako Scrum Master zespołu mogę powiedzieć, że aplikacja **Taskly** została zrobiona zgodnie z założeniami projektowymi, harmonogramem oraz wyznaczonymi User Stories. Współpraca analityka, programisty oraz testera pozwoliła na płynne przejście przez proces twórczy programu.

Perspektywy rozwoju projektu:

- Rozszerzenie scenariuszy testowych o urządzenia mobilne z systemami Android/iOS.
- Przeprowadzenie testów wydajnościowych obciążających pamięć LocalStorage przy wprowadzeniu bardzo dużej liczby rekordów jednocześnie.